



baitly

SECURITE · AUDIT DEFENSIF · CONFIDENTIEL

Audit de sécurité applicatif Baitly

PMS location courte durée — audit défensif en lecture seule, 10 phases (isolation multi-tenant, auth, injections, paiements, secrets, Kafka, dépendances, RGPD, frontend).

Date	14 juillet 2026
Périmètre	dépôt clenzy (back Java 21 / Spring Boot 3.5.13, front React 18, mobile Expo, SDK)
Findings	30 — 1 Critical, 4 High, 12 Medium, 13 Low
Verdict	NO-GO GA en l'état (bloquants = quick wins)

1. Résumé exécutif

La plateforme présente une **base de sécurité mature** : deny-by-default, validation JWT durcie (issuer + audience + signature), chiffrement PII au repos, signatures webhook de paiement correctes, montants recalculés serveur, rate limiting robuste, gestion DLT/poison Kafka, aucun secret de prod en dur ni dans l'historique git, frontend sans sink XSS.

Mais l'isolation multi-tenant — risque n°1 d'un PMS SaaS — repose sur une architecture **fragile sans filet** : **pas de RLS PostgreSQL** et le filtre Hibernate `organizationFilter` est **inerte sur les flux HTTP**. L'isolation dépend donc d'une vérification d'ownership explicite dans **chaque** endpoint. Plusieurs oublis produisent des **IDOR cross-tenant confirmés**, dont un critique.

Top 5 des risques

#	ID	Risque	Sévérité
1	F1-01	Tout utilisateur authentifié lit n'importe quelle réservation (PII guest, montants, statut paiement)	Critical · 7.7
2	F3-01	SSRF non authentifiée avec exfiltration (snapshot Playwright + sous-ressources CSS non validées)	High · 8.6
3	F1-03	Tout utilisateur authentifié initie le paiement / mute la facture d'un autre tenant	High · 8.1
4	F1-STRUCT	Pas de RLS + filtre Hibernate inerte -> aucune seconde ligne de défense ; amplifie tous les IDOR	High · 7.4
5	F1-02	Énumération cross-tenant de tout l'historique documentaire (emails, n° de factures) par un HOST	High · 7.1

Verdict : NO-GO en l'état pour une mise en production commerciale grand public

Le cluster d'IDOR cross-tenant (**F1-01** -> **F1-09**) et la **SSRF anonyme (F3-01)** sont des **bloquants** : une fuite de données entre tenants et une SSRF non authentifiée sont incompatibles avec une commercialisation SaaS. Ce sont majoritairement des **quick wins** (ajout d'une garde d'ownership déjà standardisée dans le code via `OrganizationAccessGuard`).

Go conditionnel dès que : (a) les IDOR F1-01 -> F1-09 sont gardés, (b) F3-01 est corrigé (auth + IP épinglée), (c) le déficit structurel F1-STRUCT reçoit un filet (RLS ou réactivation du filtre par transaction).

Sévérité	Nombre
Critical	1
High	4
Medium	12
Low	13
Total	30

2. Tableau des findings (trié par sévérité)

ID	Titre	Sévérité	Emplacement
F1-01	IDOR lecture — GET /api/reservations/{id}	Critical · 7.7	ReservationController.java:131
F3-01	SSRF anonyme + exfiltration (snapshot Playwright)	High · 8.6	SiteSnapshotService.java:132, 263
F1-03	IDOR écriture — POST /api/invoices/{id}/pay	High · 8.1	InvoicePaymentService.java:39
F1-STRUCT	Pas de RLS + filtre Hibernate inerte en HTTP	High · 7.4	TenantFilter.java:300
F1-02	Énumération cross-tenant historique documentaire	High · 7.1	DocumentController.java:238
F4-01	Webhook Wise fail-open sur signature (latent)	Medium · 7.5*	WiseWebhookController.java:75
F1-04	IDOR écriture — POST /api/invoices/{id}/send	Medium · 6.5	InvoicePaymentService.java:98
F6-01	Broker Kafka PLAINTEXT par défaut (pas d'auth/ACL/TLS)	Medium · 6.5	application-prod.yml:72
F6-02	Consumers Kafka de confiance -> forge d'événements	Medium · 6.5	DocumentEventService.java:44
F1-05..09	Cluster IDOR ServiceRequest + docs by-reference	Medium · ~6	ServiceRequestController.java
F1-10	Consumers Kafka sans re-scoping tenant	Medium · ~5.5	18 @KafkaListener
F3-02	DNS rebinding TOCTOU — fetch site (analyse IA)	Medium · 5.4	WebsiteFetchService.java:59
F3-06	IA : règles de confiance auto-approuvent écritures non-argent	Medium · 5.4	AgentTrustRuleService.java:44
F7-01	Gate SCA non bloquant (illusoire)	Medium · 5.3	ci-backend.yml:141
F3-03	DNS rebinding TOCTOU — webhooks sortants	Medium · 5.0	WebhookDeliveryService.java:131
F3-07	IA : remember_fact écrit sans confirmation (memory poisoning)	Medium · 5.0	RememberFactTool.java:116
F8-01	Purge de rétention désactivée en prod (PII indéfinie)	Medium · RGPD	RetentionPurgeScheduler
F8-02	Pas de droits data-subject pour les Guests/voyageurs	Medium · RGPD	GdprService.java:186
F4-02	completeTransaction sans garde de concurrence forte	Medium · 4.8	PaymentPersistence.java:191
F3-04	Source caméra RTSP/HTTP -> go2rtc sans validation	Low-Med · 4.9	CameraService.java:152
F9-01	Open redirect via successUrl/cancelUrl non validés	Low-Med · 4.7	StripePaymentProvider.java:70
F2-01	Autorisation fonctionnelle manquante (templates WhatsApp)	Low-Med · 4.3	WhatsAppTemplateController.java
F5-01	Secrets test/local commités	Low-Med · 4.0	application-local.yml:55
F3-08	IA : résultats d'outils réinjectés sans échappement	Low	AgentToolLoopRunner.java:225
F3-05	ICalUrlValidator ne bloque pas IPv6 ULA/mapped	Low · 3.7	ICalUrlValidator.java:117
F5-03	proxy_ssl_verify off sur upstream SSL	Low · 3.7	nginx.prod.conf:153
F1-11	InterventionAccessPolicy fail-open sur org NULL	Low · 3.7	InterventionAccessPolicy.java:47

ID	Titre	Sévérité	Emplacement
F2-02	Mapping de rôle fragile ADMIN/MANAGER -> SUPER_*	Low · 3.7	SecurityConfigProd.java:301
F2-03	Logout sans révocation Keycloak	Low · 3.1	AuthSessionController.java:226
F5-02	CSP unsafe-inline/unsafe-eval + incohérences en-têtes	Low · 3.1	nginx.prod.conf:53
F4-03/04	Double résa legacy / retry 200-avaleur	Low	PublicBookingService.java:1475
F6-03/F8-04	PII (email) loggée en clair (PiiMasker incohérent)	Low	DocumentEventService.java:71
F7-02/03	Vulns front non patchées / currence versions backend	Low	npm audit ; pom.xml

* CVSS de F4-01 si l'endpoint est whitelisted (actuellement neutralisé par l'absence de permitAll).

3. Détail des findings prioritaires

F1-STRUCT — Isolation 100 % applicative, sans filet (High, 7.4)

Cause racine de tout le cluster IDOR. Deux défenses théoriques sont absentes/inertes :

Aucune RLS PostgreSQL (aucune CREATE POLICY / ENABLE ROW LEVEL SECURITY dans les changesets Liquibase).

Filtre Hibernate inerte sur les flux HTTP : `open-in-view: false` sur les 4 profils, et aucun aspect ne réactive le filtre par transaction. Les 3 seuls `enableFilter` sont hors flux HTTP.

```
// TenantFilter.java:300 – le filtre echoue silencieusement au niveau servlet
private void enableHibernateOrgFilter(Long orgId) {
    try {
        Session session = entityManager.unwrap(Session.class); // IllegalStateException (open-in-view=false)
        session.enableFilter("organizationFilter").setParameter("orgId", orgId);
    } catch (IllegalStateException e) {
        logger.debug("filtre Hibernate non activable ..."); // avale -> filtre INERTE en HTTP
    }
}
```

Conséquence : chaque `findById` en flux HTTP dépend d'une garde explicite. Toute garde oubliée = fuite cross-tenant totale, sans seconde ligne de défense.

Correctif : RLS Postgres (policy `USING (organization_id = current_setting('app.current_org'))::bigint)` ou un aspect `@Around` sur `@Transactional` qui réactive le filtre depuis le TenantContext.

F1-01 — IDOR lecture GET /api/reservations/{id} (Critical, 7.7)

AV:N/AC:L/PR:L/UI:N/S:C/H/I:N/A:N

```
// ReservationController.java:131 – classe @PreAuthorize("isAuthenticated()") seulement
@GetMapping("/{id}")
public ResponseEntity<ReservationDto> getById(@PathVariable Long id) {
    Reservation r = reservationService.getByIdFetchAll(id); // WHERE r.id = :id (aucun org)
    return ResponseEntity.ok(reservationMapper.toDto(r)); // aucun validatePropertyAccess
}
```

Exploitabilité : n'importe quel utilisateur authentifié (même HOUSEKEEPER) itère `id` et lit toute réservation de tout tenant (PII guest, montants, statut paiement, codes d'accès). Les endpoints voisins (update, cancel, sendPaymentLink) appellent bien `validatePropertyAccess` — seul `getById` a été oublié.

Correctif : appeler `validatePropertyAccess(reservation.getProperty().getId(), jwt)` (garde déjà utilisée par les autres endpoints du controller).

F1-03 — IDOR écriture POST /api/invoices/{id}/pav (High, 8.1)

```
// InvoicePaymentService.java:39 – aucun controle d'org
public PaymentOrchestrationResult payInvoice(Long invoiceId, ...) {
    Invoice invoice = invoiceRepository.findById(invoiceId).orElseThrow(...); // pas de requireSameOrganization
    invoice.setPaymentTransactionId(result.transaction().getId()); // mutation cross-tenant
    invoiceRepository.save(invoice);
}
```

Tout utilisateur authentifié initie l'orchestration de paiement et mute la facture d'un autre tenant.

`sendInvoice` (F1-04) a le même défaut. Dans le même controller, `getInvoice/pdf/issue/cancel` sont org-validés. **Correctif** :

`organizationAccessGuard.requireSameOrganization(invoice.getOrganizationId(), ...)` avant toute action.

F1-02 — Énumération cross-tenant de l'historique documentaire (High, 7.1)

```
// DocumentController.java:238
@GetMapping("/generations")
```

```
@PreAuthorize("hasAnyRole('SUPER_ADMIN','SUPER_MANAGER','ADMIN','HOST')") // HOST = tenant-scoped
// -> DocumentGeneratorService.listGenerations -> findAllByOrderByCreatedAtDesc(pageable)
// AUCUN filtre organization_id, aucun post-filtre
```

Un HOST pagine l'historique de génération documentaire de tous les tenants (emails destinataires, numéros légaux de factures). L'endpoint voisin `getGenerationsByReference` post-filtre par email pour les non-staff — oublié isolé. **Correctif** : `findAllByOrganizationIdOrderByCreatedAtDesc(orgId, pageable)` .

F3-01 — SSRF non authentifiée + exfiltration (High, 8.6)

AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:N

```
// /api/public/preview-proxy/snapshot?url= (ANONYME)
private void validateUrl(String url){ ICalUrlValidator.validateAndResolve(url); } // IP épinglée JETEE
page.navigate(url, ...); // suit les 3xx + re-resout le DNS (rebinding)
String css = Jsoup.connect(href).execute().body(); // href de la page attaquante, AUCUNE validation
link.after("<style>" + css + "</style>"); // CSS interne INLINE dans la reponse -> exfiltration
```

Anonyme. Une URL publique validée qui redirige vers `169.254.169.254` / une IP interne est rendue par Chromium; et les `<link rel=stylesheet>` de la page attaquante permettent de fetch n'importe quelle URL interne et d'**inliner la réponse** dans le HTML retourné (metadata cloud, MinIO, actuator, go2rtc).

Correctif : retirer l'accès anonyme + intercepter chaque requête Playwright (Route) qui revalide l'IP + valider les href CSS via `ICalUrlValidator` et router via le pattern IP-épinglée (`PinnedSiteFetcher`).

F4-01 — Webhook Wise fail-open (Medium / High à l'activation)

```
// WiseWebhookController.java:75 - wise.public-key absente de application-prod.yml
if (wisePublicKeyPem != null && !wisePublicKeyPem.isBlank()) { ... verifie ... }
else { log.warn("verification de signature desactivee (wise.public-key vide)"); } // FAIL-OPEN
```

Un payload non signé forgé passerait un payout en PAID/FAILED. **Actuellement neutralisé** car `/api/webhooks/payouts/**` n'est pas dans le `permitAll` (403). Devient exploitable dès l'activation de Wise. Contraste : GoCardless rejette (503) si le secret est absent. **Correctif** : fail-closed.

F6-01 — Broker Kafka PLAINTEXT par défaut (Medium. 6.5)

```
# application-prod.yml:72
security.protocol: ${KAFKA_SECURITY_PROTOCOL:PLAINTEXT} # SASL/SSL cables mais desactives
sasl.jaas.config: ${KAFKA_SASL_JAAS_CONFIG:} # vide
```

Sans `SASL_SSL` en prod : aucune auth, aucun chiffrement, aucune ACL -> toute partie avec accès réseau au broker produit/consomme tous les topics. Cela rend exploitables les gaps de confiance des consumers (F6-02 : forge d'un event `documents.generate` -> document d'un autre tenant envoyé à une adresse arbitraire). Sévérité finale conditionnée à l'isolation réseau du broker (à confirmer dans clenzy-infra).

Les findings restants (F1-04 -> F1-11, F2-*, F3-02 -> F3-09, F4-02 -> F4-04, F5-*, F6-03, F7-*, F8-*, F9-01) sont détaillés dans le rapport Markdown avec fichier: ligne et correctifs.

4. Plan de remédiation priorisé

Quick wins (< 1 jour) — bloquants GA

F1-01 → F1-09 : ajouter la garde `requireSameOrganization(entity.getOrganizationId())` (déjà standardisée) sur chaque endpoint IDOR : réservations `getById`, invoices `pay/send`, cluster `ServiceRequest`, documents `generations/by-reference`.

F3-01 : retirer l'accès anonyme au snapshot + IP épinglée sur la page ET les sous-ressources CSS.

F4-01 : rendre la vérif de signature Wise obligatoire (fail-closed) avant toute activation.

Court terme (< 1 semaine)

F1-STRUCT : introduire un filet — RLS Postgres ou réactivation du filtre Hibernate par transaction. Le changement le plus structurant : sans lui, chaque nouvel endpoint reste un IDOR potentiel.

F6-01/F6-02 : activer SASL_SSL + ACL Kafka ; enrober les consumers via `TenantScopedExecutor` + revalider l'appartenance des `referenceld` / escrow-réservation.

F3-02/03/04 : router les fetch/webhooks sortants via le pattern IP-épinglée ; allow-list schéma/hôte pour les sources caméra.

F3-06/07 : élargir la liste non-auto-approuvable IA aux tools de communication/annulation ; reclasser `remember_fact` en écriture tracée.

F7-01 : rendre le gate SCA bloquant (retirer `|| true` / `continue-on-error`, `failBuildOnCVSS=7`) ; npm audit fix.

F4-02/03 : contrainte unique `idempotency_key` + UPDATE conditionnel ; index unique `stripe_session_id`.

Structurel / conformité

F8-01/02 : activer la purge de rétention en prod (durées légales par catégorie) ; étendre l'effacement/export aux Guest/GuestDeclaration.

F8-03 : confirmer DPA/SCCs avec les sous-traitants LLM US (Anthropic/OpenAI/Voyage) ; minimiser la PII dans les prompts.

F5-01 : révoquer/roter les secrets test commités, externaliser.

F5-02/03, F2-*, F9-01, F6-03 : durcir CSP (nonces) ; confirmer `proxy_ssl_verify` ; allow-list rôle templates WhatsApp + mapping de rôle strict + révocation logout ; allow-list URLs de retour paiement ; masquage PII en logs.

Hors-repo (clenzy-infra / clenzy-sites) : ACL bucket OVH/MinIO privé ; secrets docker-compose ; sanitisation + CSP du SSR des sites publiés.

5. Points positifs (à préserver)

Deny-by-default (`anyRequest().denyAll()`) ; profils de sécurité en liste positive + EnvironmentValidator fail-fast (pas de admin/admin, JASYPT requis).

Validation JWT durcie : issuer + audience (clenzy-api) + signature JWKS + timestamps → rejette cross-realm ET cross-client.

Résolution tenant fail-closed, dérivée du JWT seul ; garde centralisée OrganizationAccessGuard (fail-closed) largement utilisée.

Paiements : signatures webhook correctes (Stripe/CMI/Attijari/PayTabs/Payzone), montants recalculés serveur (cross-check 400), StripeAmounts.toMinorUnits (HALF_UP), transitions gardées (CAS), secrets chiffrés per-org, aucun log de PAN/CVV/secret.

Pas d'injection SQL ; import iCal SSRF-safe (IP épinglée) ; scoping tenant des agents IA solide (le LLM ne fournit jamais l'org ; RAG scopé org ; tools argent jamais auto-approuvés ; HITL structurel).

Rate limiting robuste (Lua atomique, brute-force, IP anti-spoofing) ; uploads validés ; DLT/poison Kafka robustes.

Cookies BFF HttpOnly/Secure/SameSite=Strict ; frontend sans sink XSS (HTML non fiable sandboxé) ; tokens mobile en SecureStore ; PII chiffrée au repos ; en-têtes/TLS nginx solides ; aucun secret de prod en dur ni dans l'historique git.